

Datatype

September 8, 2026

1 TUPLE

Tuple is immutable

```
[6]: Tuple = (0,1,2,3,4,5)
```

```
[7]: Tuple[3]
```

```
[7]: 3
```

```
[8]: Tuple[:3]
```

```
[8]: (0, 1, 2)
```

```
[9]: Tuple[1:6:2]
```

```
[9]: (1, 3, 5)
```

```
[10]: Tuple[0] = 10
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[10], line 1  
----> 1 Tuple[0] = 10  
  
TypeError: 'tuple' object does not support item assignment
```

```
[11]: Tuple[5] + 5
```

```
[11]: 10
```

```
[12]: 5 in Tuple
```

```
[12]: True
```

```
[13]: 1 not in Tuple
```

```
[13]: False
```

```
[14]: Tuple2 = (5,6,7)
```

```
[15]: Tuple = (0,1,2,3,4,5)
      Tuple2 = (5,6,7)
      print(Tuple + Tuple2)
```

```
(0, 1, 2, 3, 4, 5, 5, 6, 7)
```

```
[16]: tuple1e = (10,) #single element tuple should end with comma
```

```
[17]: test =(1,[1,2])
```

```
[18]: test[1] = [1,2,3]
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[18], line 1
----> 1 test[1] = [1,2,3]

TypeError: 'tuple' object does not support item assignment
```

```
[19]: test[1][1] = [1,2,3]
```

```
[20]: test
```

```
[20]: (1, [1, [1, 2, 3]])
```

2 LIST

List is immutable

```
[21]: num1 = [1,2,3,4,5,6]
      str1 = ['a','b','c']
      mix1 = [1,2,6,45,'a','b'] # Mix datatype list
```

```
[22]: num1[2] is 10
```

```
<>:1: SyntaxWarning: "is" with 'int' literal. Did you mean "=="?
<>:1: SyntaxWarning: "is" with 'int' literal. Did you mean "=="?
C:\Users\AV\AppData\Local\Temp\ipykernel_19876\4213587995.py:1: SyntaxWarning:
"is" with 'int' literal. Did you mean "=="?
      num1[2] is 10
```

```
[22]: False
```

```
[23]: str1[0] in mix1
```

[23]: True

```
[24]: str1[0] = 'b'  
      str1[1:] = ['c','d']
```

```
[25]: str1
```

[25]: ['b', 'c', 'd']

```
[26]: str1.insert(0,'a')
```

```
[27]: str1
```

[27]: ['a', 'b', 'c', 'd']

```
[28]: str1.append('e')
```

```
[29]: str1
```

[29]: ['a', 'b', 'c', 'd', 'e']

```
[30]: str12 = ['f','g','h','i','i']
```

```
[31]: str1.extend(str12)
```

```
[32]: str1
```

[32]: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'i']

```
[33]: str1.remove('i')
```

```
[34]: str1
```

[34]: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i']

```
[35]: str1.pop(2)
```

[35]: 'c'

```
[36]: str1
```

[36]: ['a', 'b', 'd', 'e', 'f', 'g', 'h', 'i']

```
[37]: del str12[2]  
      print (str12)
```

['f', 'g', 'i', 'i']

```
[38]: str12.clear()
```

```
[39]: strl2
```

```
[39]: []
```

```
[40]: del strl2
```

```
[41]: strl2
```

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[41], line 1  
----> 1 strl2  
  
NameError: name 'strl2' is not defined
```

2.1 Challenge

Even no from list

```
[42]: nol = [1,2,3,4,5,6,7,8,9,10]
```

```
[43]: for i in nol:  
        if i % 2 == 0:  
            print(i, end=" ")
```

2 4 6 8 10

```
[44]: # List Comprehension  
even = [x for x in nol if x % 2 == 0]  
print (even)
```

[2, 4, 6, 8, 10]

Listt Comprehention

['var' for 'var' in 'LIST' 'Condition']

2.2 Challenge

Leap Year from list

```
[45]: year = [2000,1900,2024,2004,2026]
```

```
[46]: for i in year:  
        if i % 100 ==0:  
            if i % 400 == 0:  
                print(i, "is a leap year")  
            else:  
                print(i, "is a not leap year")
```

```
else:
    if i % 4 == 0:
        print(i, "is a leap year")
    else:
        print(i, "is a not leap year")
```

```
2000 is a leap year
1900 is a not leap year
2024 is a leap year
2004 is a leap year
2026 is a not leap year
```

[]: